DLOps Assignment 5

LoRA Fine-tuning + Adversarial Attacks

Name	Akshat Jain
Roll Number	M25CSA003
WandB Q1	https://wandb.ai/akshst08092001-iit-jodhpur/DLOps-Ass5-Q1
WandB Q2	https://wandb.ai/akshst08092001-iit-jodhpur/DLOps-Ass5-Q2
HuggingFace	https://huggingface.co/Despo08/vit-lora-cifar100-best
GitHub	https://github.com/m25csa003-glitch/MLOps-AkshatJain-M25CSA003/tree/Assignment-5

Hardware & Environment

Component	Details
GPU	NVIDIA A30 (24GB VRAM), CUDA 12.8
CPU	College HPC Cluster (cn02)
Dev Machine	Apple MacBook Air M4 (16GB RAM)
Container	Singularity 4.3.1 (Docker: pytorch/pytorch:2.1.0-cuda11.8)
Framework	PyTorch 2.1.0, PEFT 0.10.0, IBM ART 1.17.0

Q1: ViT-S + LoRA Fine-tuning on CIFAR-100

Approach

A pretrained ViT-Small (patch16, 224) model from ImageNet was fine-tuned on CIFAR-100 (100 classes). Two strategies were compared: (1) head-only fine-tuning (baseline), and (2) LoRA injected into Q, K, V attention weights using PEFT library. LoRA enables parameter-efficient adaptation with very few trainable parameters. Optuna was used for hyperparameter optimization.

Docker/Singularity Setup

Training was performed inside a Singularity container built from the official PyTorch Docker image (pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime). The Dockerfile is included in the repository. On the HPC cluster, Docker was not available, so Singularity was used as the container runtime, which is the standard approach for HPC environments.

Experiment 1: Baseline (No LoRA) — Head Only

Trainable Params: 38,500 | Test Accuracy: 80.92%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	1.0826	0.7449	71.44%	78.16%
2	0.6454	0.6841	80.88%	80.02%
3	0.5684	0.6769	82.81%	79.91%
4	0.5211	0.6673	84.00%	80.62%
5	0.4768	0.6557	85.22%	80.54%
6	0.4458	0.6678	86.14%	80.62%
7	0.4179	0.6560	87.01%	80.92%
8	0.3976	0.6482	87.45%	81.23%
9	0.3800	0.6475	88.06%	81.41%
10	0.3715	0.6311	88.38%	81.52%

Experiment 2: Rank=2, Alpha=2

Trainable Params: 75,364 | Test Accuracy: 87.99%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.4514	0.8823	46.80%	77.63%
2	0.6531	0.5619	82.44%	83.79%
3	0.4810	0.4831	85.97%	85.28%
4	0.4183	0.4444	87.45%	86.17%
5	0.3850	0.4290	88.33%	86.72%
6	0.3590	0.4166	89.00%	87.14%
7	0.3418	0.4067	89.45%	87.28%
8	0.3331	0.4097	89.65%	87.21%
9	0.3263	0.4089	89.95%	87.15%
10	0.3214	0.3980	89.96%	87.64%

Experiment 3: Rank=2, Alpha=4

Trainable Params: 75,364 | Test Accuracy: 88.19%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.3071	0.7885	49.52%	79.99%
2	0.6045	0.5304	83.38%	85.00%
3	0.4637	0.4622	86.18%	86.35%
4	0.4072	0.4348	87.81%	86.93%
5	0.3723	0.4186	88.53%	87.45%
6	0.3489	0.4031	89.12%	87.59%
7	0.3309	0.3959	89.63%	87.72%
8	0.3228	0.3942	90.04%	88.10%
9	0.3171	0.4000	90.03%	87.75%
10	0.3138	0.3983	90.19%	87.81%

Experiment 4: Rank=2, Alpha=8

Trainable Params: 75,364 | Test Accuracy: 88.28%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.2608	0.7609	50.99%	80.62%
2	0.5880	0.5283	83.61%	84.83%
3	0.4531	0.4625	86.58%	86.28%
4	0.4016	0.4328	87.66%	86.92%
5	0.3631	0.4136	88.93%	87.62%
6	0.3362	0.4031	89.55%	87.78%
7	0.3198	0.4021	90.00%	87.57%
8	0.3113	0.3972	90.36%	88.20%
9	0.3034	0.3939	90.51%	88.16%
10	0.2996	0.3981	90.51%	87.92%

Experiment 5: Rank=4, Alpha=2

Trainable Params: 112,228 | Test Accuracy: 88.30%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.4669	0.8639	46.76%	78.40%
2	0.6345	0.5537	82.81%	83.85%
3	0.4721	0.4717	86.00%	85.94%
4	0.4112	0.4459	87.56%	86.35%
5	0.3761	0.4218	88.37%	87.05%
6	0.3536	0.4105	89.12%	87.32%
7	0.3386	0.4140	89.40%	87.19%
8	0.3295	0.4037	89.63%	87.54%

9	0.3207	0.3970	90.02%	87.49%
10	0.3165	0.4058	90.17%	87.69%

Experiment 6: Rank=4, Alpha=4

Trainable Params: 112,228 | Test Accuracy: 88.05%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.3471	0.7819	49.02%	80.04%
2	0.5935	0.5267	83.66%	84.83%
3	0.4570	0.4630	86.45%	86.15%
4	0.4016	0.4470	87.66%	86.05%
5	0.3672	0.4174	88.73%	87.31%
6	0.3462	0.4105	89.10%	87.23%
7	0.3309	0.4038	89.42%	87.50%
8	0.3214	0.3956	89.78%	87.75%
9	0.3111	0.3941	90.36%	88.00%
10	0.3092	0.3988	90.39%	87.73%

Experiment 7: Rank=4, Alpha=8

Trainable Params: 112,228 | Test Accuracy: 88.61%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.1792	0.7361	52.01%	80.64%
2	0.5747	0.5228	83.81%	84.78%
3	0.4429	0.4575	86.87%	86.43%
4	0.3906	0.4382	88.09%	86.97%
5	0.3604	0.4125	88.91%	87.19%
6	0.3335	0.3963	89.55%	87.86%
7	0.3170	0.3959	90.12%	87.90%
8	0.3063	0.3893	90.45%	88.13%
9	0.2985	0.3910	90.64%	88.00%
10	0.2925	0.3882	90.87%	88.12%

Experiment 8: Rank=8, Alpha=2

Trainable Params: 185,956 | Test Accuracy: 88.07%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.4157	0.8471	47.96%	78.76%
2	0.6312	0.5438	82.49%	84.10%
3	0.4735	0.4738	85.92%	85.48%
4	0.4143	0.4362	87.48%	86.57%
5	0.3780	0.4314	88.39%	86.58%
6	0.3536	0.4185	88.92%	86.98%

7	0.3410	0.4089	89.35%	87.35%
8	0.3279	0.4028	89.89%	87.26%
9	0.3244	0.3959	89.77%	87.60%
10	0.3227	0.4007	90.08%	87.33%

Experiment 9: Rank=8, Alpha=4

Trainable Params: 185,956 | Test Accuracy: 88.39%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.3654	0.8024	49.21%	79.61%
2	0.6071	0.5331	83.30%	84.23%
3	0.4578	0.4660	86.46%	85.84%
4	0.4013	0.4338	87.66%	86.57%
5	0.3675	0.4154	88.74%	87.13%
6	0.3448	0.4072	89.38%	87.30%
7	0.3267	0.3983	89.83%	87.54%
8	0.3156	0.3985	90.30%	87.71%
9	0.3101	0.3952	90.39%	87.66%
10	0.3077	0.3958	90.47%	87.82%

Experiment 10: Rank=8, Alpha=8

Trainable Params: 185,956 | Test Accuracy: 88.70%

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	2.2525	0.7477	50.83%	80.54%
2	0.5732	0.5140	84.02%	84.85%
3	0.4410	0.4504	86.85%	86.68%
4	0.3871	0.4267	88.25%	87.26%
5	0.3524	0.4045	89.17%	87.87%
6	0.3304	0.4016	89.77%	87.70%
7	0.3116	0.3901	90.24%	88.17%
8	0.3068	0.3904	90.39%	88.04%
9	0.2927	0.3886	90.68%	87.82%
10	0.2925	0.3861	90.83%	88.15%

Optuna Hyperparameter Search

Optuna was used to search over rank (2,4,8,16), alpha (2,4,8,16), dropout (0.0-0.3), and learning rate (1e-5 to 1e-3) with 20 trials and MedianPruner.

Best Config Found: Rank=4, Alpha=16, Dropout=0.2, LR=0.000504, Val Accuracy=89.07%

Best Config Training (Rank=4, Alpha=16, Dropout=0.2)

Trainable Params: 112,228 | Test Accuracy: 89.96% (BEST)

Epoch	Train Loss	Val Loss	Train Acc	Val Acc
1	0.8650	0.4408	77.39%	86.69%
2	0.3514	0.4059	89.02%	87.41%
3	0.2799	0.3921	91.07%	88.31%
4	0.2269	0.3950	92.70%	88.09%
5	0.1880	0.3904	93.90%	88.59%
6	0.1565	0.3818	94.96%	88.65%
7	0.1341	0.3792	95.80%	88.84%
8	0.1155	0.3804	96.44%	88.99%
9	0.1033	0.3731	96.97%	89.12%
10	0.0965	0.3763	97.21%	89.03%

Q1 Testing Summary Table

LoRA	Rank	Alpha	Dropout	Test Acc	Trainable Params
No	-	-	-	80.92%	38,500
Yes	2	2	0.1	87.99%	75,364
Yes	2	4	0.1	88.19%	75,364
Yes	2	8	0.1	88.28%	75,364
Yes	4	2	0.1	88.30%	112,228
Yes	4	4	0.1	88.05%	112,228
Yes	4	8	0.1	88.61%	112,228
Yes	8	2	0.1	88.07%	185,956
Yes	8	4	0.1	88.39%	185,956
Yes	8	8	0.1	88.70%	185,956
Yes (Optuna)	**4**	**16**	**0.2**	**89.96%**	**112,228**

Analysis

LoRA significantly outperforms the baseline (80.92%) across all configurations, achieving up to 88.70% with Rank=8, Alpha=8. The Optuna best config (Rank=4, Alpha=16, Dropout=0.2) achieves the highest test accuracy of 89.96% while using only 0.51% trainable parameters (112,228 out of 21.8M). This demonstrates the effectiveness of LoRA for parameter-efficient fine-tuning. Higher rank generally improves performance but with diminishing returns. The gradient norms logged during training show stable learning dynamics across all LoRA layers.

Q2: Adversarial Attacks using IBM ART

Part (i): FGSM Attack — From Scratch vs IBM ART

A ResNet18 model was trained from scratch on CIFAR-10 achieving 93.41% test accuracy (target >= 72%). FGSM attacks were then implemented both from scratch and using IBM ART.

ResNet18 Training Results

Test Accuracy: 93.41% | Target (>=72%): PASSED

FGSM Attack Comparison

Epsilon	Clean Acc	FGSM Scratch	FGSM ART	Acc Drop (Scratch)	Acc Drop (ART)
0.01	93.41%	39.89%	44.40%	53.52%	49.01%
0.02	93.41%	32.18%	37.70%	61.23%	55.71%
0.05	93.41%	23.84%	25.80%	69.57%	67.61%
0.10	93.41%	12.49%	14.10%	80.92%	79.31%
0.15	93.41%	10.62%	~11.30%	82.79%	82.11%
0.20	93.41%	10.10%	-	83.31%	-
0.30	93.41%	10.00%	-	83.41%	-

FGSM Analysis

Both FGSM implementations (scratch and IBM ART) produce similar results. At epsilon=0.01, accuracy drops drastically from 93.41% to ~40%, demonstrating that even small perturbations can fool the model. IBM ART's FGSM consistently achieves slightly higher adversarial accuracy than the scratch implementation, likely due to better numerical precision in gradient computation. At higher epsilon values (>=0.15), both methods reduce accuracy to near-random (~10%), indicating the model is completely fooled.

Part (ii): Adversarial Detection Models

ResNet34-based binary classifiers were trained to detect adversarial examples generated by PGD and BIM attacks using IBM ART.

Attack	Detection Accuracy	Target	Status	Trainable Params
PGD	**99.33%**	>= 70%	**PASSED**	~21.8M
BIM	**99.53%**	>= 70%	**PASSED**	~21.8M

Detector Analysis

Both detectors achieve near-perfect detection accuracy (~99%), far exceeding the 70% target. The detectors converge quickly — PGD detector reaches >99% by epoch 6, and BIM detector by epoch 4. This high accuracy suggests that adversarial examples created by PGD and BIM leave detectable artifacts in the feature space. PGD (40 iterations) creates stronger perturbations than BIM (10 iterations), yet both are highly detectable. The ResNet34 architecture is well-suited for this binary detection task.

Links & Submission

Resource	Link
WandB Q1	https://wandb.ai/akshst08092001-iit-jodhpur/DLOps-Ass5-Q1
WandB Q2	https://wandb.ai/akshst08092001-iit-jodhpur/DLOps-Ass5-Q2
HuggingFace Model	https://huggingface.co/Despo08/vit-lora-cifar100-best
GitHub Branch	https://github.com/m25csa003-glitch/MLOps-AkshatJain-M25CSA003/tree/Assignment-5

Note on Docker Usage

As per assignment requirements, all experiments were performed using a container. A Dockerfile (pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime base) is provided in the GitHub repository. On the college HPC cluster, Docker was unavailable, so Singularity 4.3.1 was used to run the Docker image — this is the standard approach for HPC environments and satisfies the container requirement.

Note on WandB Logging

WandB runs were initially logged in offline mode due to institutional firewall restrictions. All runs were subsequently synced to the cloud. The WandB project 'DLOps-Ass5-Q1' contains 11 runs (baseline + 9 LoRA combinations + Optuna best config) with train/val loss curves, accuracy graphs, classwise histograms, and gradient norm plots for all LoRA layers.